

Exploratory Test 1 - Intercity Ticket Search

Objective: Identify potential issues in the ticket search and booking module

Duration: 10 minutes

Scope: "origin station" and "destination station" fields, date, buttons, number of adult and child passengers

- The maximum number of passengers in first and second class is 6.
- A child ticket cannot be added when 6 adults are selected.
- The lack of clear information about this limitation may mislead the user.

Actual result: After reaching a total of 6 passengers, the + buttons for adults and children become inactive. The user is not informed that the maximum passenger limit has been reached.

Expected result: After reaching the limit of 6 passengers, the system should display a message, for example: "The maximum number of passengers in a single booking is 6."

Conclusion: The user may not understand why a child cannot be added to the booking, which may cause confusion or lead to abandoning the booking process.

Additional observation: In other parts of the booking process, the system displays a message explaining why an option is unavailable, for example when adding a bicycle ticket. No similar information is displayed when the maximum limit of 6 passengers is reached.

Severity: Medium

Priority: Medium

Project: Login Page Testing – Banking Application X

Project Description: The goal of the tests was to verify the correct operation of the login module in a banking application. The focus was on the "Login" and "Password" fields, the "Log in" button, error messages, and user session behavior.

Test Type: Checklist (Manual Testing)

Test Table:

Test Checklist – Login Module (Banking Application X)

ID	Element	Check	Status
CH-01	"Login" Field	Accepts a valid login	PASS
CH-02	"Login" Field	Rejects an empty login	PASS
CH-03	"Login" Field	Rejects login with invalid characters	PASS
CH-04	"Login" Field	Accepts maximum login length	PASS
CH-05	"Password" Field	Password is masked	PASS

ID	Element	Check	Status
CH-06	“Password” Field	Rejects an empty password	PASS
CH-07	“Password” Field	Accepts maximum password length	PASS
CH-08	“Password” Field	Accepts special characters	PASS
CH-09	“Log in” Button	Works with valid credentials	PASS
CH-10	“Log in” Button	Disabled when login is empty	PASS
CH-11	“Log in” Button	Disabled when password is empty	PASS
CH-12	“Log in” Button	Responds to Enter key in password field	PASS
CH-13	Error Message	Displays for incorrect password	FAIL
CH-14	Error Message	Displays for non-existent login	FAIL
CH-15	Error Message	Is clear to the user	PASS
CH-16	“Forgot Password” Link	Redirects to password reset	PASS
CH-17	“Register” Link	Redirects to registration form	PASS
CH-18	“Password” Field	Password visibility toggle works	PASS
CH-19	“Login” Field	Autofocus works on page load	PASS
CH-20	Session	User stays logged in after refresh	PASS
CH-21	Session	Session ends after logout	PASS
CH-22	Form Fields	Field labels are visible	PASS
CH-23	Responsiveness	Page works on mobile	PASS
CH-24	Responsiveness	Page works on tablet	PASS
CH-25	“Log in” Button	Changes state after clicking	PASS
Conclusions:			

1. The login page works correctly in most scenarios.
2. Incorrect messages were observed for incorrect passwords and non-existent logins (FAIL).
3. Testing identified areas for improvement and verified the basic functionality and usability of the login module.

Test Scope:

- Fields: login, password
- Validations
- Responsiveness
- User session

Test Environment:

- Browser: Chrome
- System: Windows
- Test Resolution: 1920x1080

A total of 25 tests were performed: 23 resulted in PASS and 2 in FAIL (the error messages require improvement).

Project: API Testing – FakeStoreAPI (Postman)

Project Description: As part of manual testing, I used the public FakeStoreAPI. The goal was to test basic endpoints of the sample store: retrieving the product list (GET) and adding a new product (POST). Response correctness, data structure, and error handling were verified.

Tool: Postman

API Test Table

ID	Endpoint	Method	Check	Status
API-01	/products	GET	Status 200 OK and product list returned	PASS
API-02	/products	GET	Products contain the expected fields: id, title, price, category, description, image	PASS
API-03	/products	POST	Create a product using valid data	PASS
API-04	/products	POST	Status 201 Created and generated ID returned	PASS
API-05	/products	POST	Send a request with incomplete product data	FAIL
API-06	/products	GET	Verify response after creating a product	PASS

Description of Tests Performed in Postman

1. GET /products – retrieved the list of all products. Verified:
 - response status (200 OK)
 - correct JSON response structure
 - presence of required fields (id, title, price, category, description, image)
 2. POST /products – added a new product. Verified:
 - sending valid data
 - response status (201 Created)
 - whether the returned product has a unique ID
 3. Negative test (POST /products) – a request with incomplete product data was sent to verify how the API handles invalid input.
-

Conclusions

- The API correctly handles standard GET and POST requests.
- The portfolio demonstrates practical skills in creating and executing requests in Postman, verifying API responses, and documenting test results.
- GET and POST tests verify both positive and negative API scenarios.
- The negative test showed that the API accepts a POST request with incomplete data instead of returning a validation error. In a production API, this behavior could result in incomplete data being stored.

Screenshot 1 - GET /products (Product List)

The screenshot shows the server response to a GET request sent to the /products endpoint. Response status: 200 OK. The received data has a valid JSON structure and contains the required fields (id, title, price, description, category, image). The test confirms that the API correctly returns a list of available products.

Screenshot 2 – POST /products (Adding a Product)

The screenshot shows the result of sending a POST request with a valid JSON body. Response status: 201 Created. The API returned a newly created product with an automatically generated ID field. The test confirms that the endpoint correctly handles adding new products.

Body request (POST /products)

```
{
  "title": "Kitchen Table",
  "price": 99.99,
  "description": "Large Wooden Table",
  "category": "kitchen",
  "image": "https://example.com/stol.jpg"
}
```

Screenshot 3 – Retrieving the Added Product by ID (GET /products/{id})

The screenshot shows a GET request for a product with an ID generated during the POST test. Response status: 200 OK. The server response contains the same data that was previously sent in the POST request. The test confirms that the GET /products/{id} endpoint correctly returns a product for an existing identifier.

Example JSON of a randomly retrieved product:

```
{
```

```
"id": 1,

"title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",

"price": 109.95,

"description": "Your perfect pack for everyday use and walks in the forest. Stash your laptop
(up to 15 inches) in the padded sleeve, your everyday",

"category": "men's clothing",

"image": "https://fakestoreapi.com/img/81fPKd-2AYL. AC SL1500 t.png"

}
```

Bug Report – JIRA (Example)

>

Wyszukiwarka

>

Lista połączeń

>

Twoja podróż

>

Podsumowanie

>

Płatność

>

Bilet

Czw. 23.07.2026 19:48 > 23:19
Gdańsk Gł. > Warszawa Centr.
 53170 Karpaty

6x Pasażerów 6x Bilet Normalny ZMIEN

PREFERENCJE WYBORU MIEJSCA

WYBIERZ MIEJSCE

BILETY DODATKOWE

Rower
9,10 zł

- 0 +

Pies
15,20 zł

- 6 + 91,20 zł

Bagaż
5,10 zł

- 6 + 30,60 zł

Aby dodać bilet na przewóz roweru, wybierz miejsce dla rowerzysty na schemacie miejsc.

